

Corso: Algoritmi e strutture dati

Quiz: ASD2026 - II Appello - Programmazione

	Marco Pedron s285048
Iniziato	20 febbraio 2026, 12:30
Stato	Completato
Terminato	20 febbraio 2026, 14:10
Tempo impiegato	1 ora 40 min.
Valutazione	2,75 su un massimo di 30,00 (9%)
Riepilogo del tentativo	i i 1 2 3 4 5 i 6 7 8 9

Informazione

Attenzione!

Indicare quale tipologia di esame si intenda svolgere rispondendo alla domanda a scelta multipla seguente (domanda 1).

Si noti che è possibile leggere tutte o parte delle domande prima di effettuare la scelta e rispondere alla domanda 1.

Le domande dalla 3 alla 5 sono dedicate all'esame semplificato (traccia da 12pt). In particolare:

- la domanda 3 è il primo esercizio (4 punti).
- la domanda 4 è il secondo esercizio (3 punti).
- la domanda 5 è il terzo esercizio (5 punti).

Le domande dalla 7 alla 9 sono dedicate all'esame completo (traccia da 18pt). In particolare:

- la domanda 7 fa parte dell'esercizio sulla struttura dati
- la domanda 8 fa parte dell'esercizio sul problema di verifica
- la domanda 9 fa parte dell'esercizio sul problema di ottimizzazione

Un apposito separatore fa da intervallo tra le domande del compito da 12pt e le domande del compito da 18pt.

Informazione

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti)

- se non indicato diversamente, è consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne
- gli header file riferiti dal codice dovranno essere inclusi nella versione del programma allegata alla relazione
- i modelli delle funzioni ricorsive **non** sono considerati funzioni standard
- consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro il 23/02/2026, alle ore 23:59, mediante caricamento su Portale. Per i laureandi che intendano presentare domanda di laurea nella sessione di marzo 2026, la scadenza è il 21/2 alle 23:59.
- le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale.
- QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERRÀ APPLICATA UNA PENALIZZAZIONE. Non è necessario includere nella relazione esercizi o parti di esercizi completamente omesse in sede di esame. Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto durante l'appello. Non verranno corretti i compiti di cui non sarà stata inviata la relazione nei tempi stabiliti.

Domanda 1

Completo

Non valutata

Indicare quale tipologia di esame si intende svolgere, selezionando UNA sola scelta tra a (12 punti) e b (18 punti).

E' possibile, in aggiunta, un'ulteriore opzione c, da selezionare SOLO nel caso di laureando/a che intenda (e possa) presentare domanda di laurea per la sessione di MARZO 2026.

Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo eventuale orale, CARICARE LA RELAZIONE NEGLI ELABORATI entro il 21/2 alle 23:59.

Scegli una o più alternative:

- ☒ a. 12 Punti - Semplificato
- ☐ b. 18 Punti - Completo
- ☒ c. Selezionare SOLO nel caso di laureando/a che intenda (qualora si superi l'esame) presentare domanda di laurea nella sessione di MARZO 2026.
Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo orale. CARICARE LA RELAZIONE entro il 21/2 alle 23:59.

Risposta parzialmente esatta.

Ignorare il feedback di questa domanda.

Le risposte corrette sono: 12 Punti - Semplificato, 18 Punti - Completo, Selezionare SOLO nel caso di laureando/a che intenda (qualora si superi l'esame) presentare domanda di laurea nella sessione di MARZO 2026.

Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo orale. CARICARE LA RELAZIONE entro il 21/2 alle 23:59.

Domanda 2

Completo

Non valutata

Traccia da 12 punti:



È possibile rispondere alle 3 domande di questa pagina su Visual Studio Code, accessibile tramite la piattaforma CrownLabs.

Lasciare vuote le caselle di testo, se si sceglie di usare questa soluzione.

Aprire solo la piattaforma Crownlabs della traccia scelta, non entrambe!

La risposta a questa domanda è stata inserita tramite un'istanza Crownlabs ed è visibile sul Portale della Didattica sulla sezione **Consegna Elaborati** di questo insegnamento.

Domanda 3

Completo

Punteggio
ottenuto 1,25
su 4,00

Sono dati il quasi ADT Item, il tipo Key e le funzioni KEYcmp e ITEMprint, definiti sotto:

```
#define MAXS 40
typedef struct {
    char name[MAXS];
    float val;
} Item;
typedef char *Key;
int KEYcmp(Key k1, Key k2) {
    return strcmp(k1, k2);
}
void ITEMprint(Item *item) {
    printf("name: %s, val: %f\n", item->name, item->val);
}
```

Sapendo che il campo name è la chiave, si scriva la funzione KEYget, che, dato un Item, ne ritorni la chiave.

Si realizzi poi la funzione ProcessItems che, dato un vettore di Item, ritorni 3 risultati

A. una stringa dinamica contenente la concatenazione delle chiavi presenti nel vettore (la stringa va allocata esattamente della dimensione necessaria, senza sovrallocazione),

- B. il puntatore all'item contenente la chiave più grande (usando KEYcmp per i confronti)
- C. la somma di tutti i campi val nel vettore.

La funzione deve poter essere chiamata da un main come il seguente.

```
int main (void) {
    Item vet[]={{"Parigi",2.2},{ "Roma",2.75},{ "Londra",8.8},{ "Berlino",3.75}};
    Item *maxp;
    char *keys; float sumVal;
    int n=sizeof(vet)/sizeof(Item);
    sumVal = ProcessItems(vet,n,&keys,&maxp);
    printf("La somma dei valori e': %f\n", sumVal);
    printf("La concatenazione delle stringhe e': %s\n", keys);
    printf("Il massimo e': ");
    ITEMprint(maxp);
    free(keys);
    return 0;
}
```

Con il main riportato, l'output sarebbe:

```
La somma dei valori e': 17.500000
La concatenazione delle stringhe e': ParigiRomaLondraBerlino
Il massimo e': name: Roma, val: 2.750000
```

Commento:

KEYget

Sbaglia a passare Item invece che puntatore ad Item. Il puntatore ritornato non è valido (rivedere la lezione sugli ADT Item).

ProcessItems

A) parametri: ok

B) allocazione: NO. Alloca (sovrallocato) per una stringa, poi rialloca alla fine, dopo aver già scritto tutto in memoria non allocata.

C) concatenazione: ok la copia ma non il conteggio dei caratteri (sizeof non serve in casi come questi, serve strlen).

D) ritorno risultato e stringa dinamica: li sbaglia entrambi: k= dovrebbe essere *k= e &max punta a unItem variabile locale, non al vettore di item.

E) altro (massimo, somma): ok

Domanda 4

Completo

Punteggio
ottenuto 0,50
su 3,00

È dato un tipo BST (ADT di prima classe), avente nodi estesi in modo tale da contenere puntatore al padre e dimensione del sottoalbero. Il tipo Item (quasi ADT) e le struct BSTnode (con puntatore link) e binarysearchtree sono definiti come segue

```
typedef struct {
    char name[MAXC];
} Item;
typedef struct BSTnode* link;
struct BSTnode {Item item; link p; link l; link r; int N; };
struct binarysearchtree { link root; link z; };
typedef struct binarysearchtree *BST;
```

Si scriva una funzione che, dato un BST b, generi un duplicato (un BST con la stessa topologia, contenente un duplicato dei dati).

La funzione abbia prototipo

```
BST BSTdup(BST b);
```

Se utile per la soluzione, si possono richiamare funzioni spiegate a lezione.

Commento:

Purtroppo lo schema usato è completamente errato: immagina nei commenti di riuscire a scendere e risalire in modo iterativo (all'interno della funzione, ma questo si può fare solo ricorsivamente).

Poi chiama una BSTinsert_leafR che non garantisce per nulla di mantenere la topologia.

Consiglio decisamente di guardare la (semplice) soluzione proposta.

Domanda 5

Completo

Punteggio
ottenuto 1,00
su 5,00

Si vogliono contare i numeri in una data base (**base**, compresa tra 2 e 10), aventi un dato numero di cifre (**nDigits**) tali da rispettare i seguenti vincoli:

- A. ogni cifra può comparire al più **maxRip** volte nel numero
- B. la differenza (in valore assoluto) tra due cifre consecutive può essere al massimo 2
- C. il numero di cifre distinte (senza tener conto delle ripetizioni) deve essere almeno pari a $nDigits/2$ (divisione tra interi)

Ad esempio, con base 8, 7 cifre, massimo numero di ripetizioni 3, il numero 5356533 rispetta i vincoli, 3232224 non rispetta il primo vincolo, 5376551 non rispetta il secondo, 3223322 non rispetta il primo e il terzo.

La funzione da realizzare ritorna come risultato il conteggio dei numeri che rispettano i vincoli. Il prototipo è

```
int cntNum(int base, int nDigits, int maxRip);
```



Commento:

Troppo poco: c'è solo un wrapper che alloca i vettori dinamici, e dei commenti che indicano l'utilizzo di un modello errato.

Mi spiace ma in questo esame tutti gli esercizi sono decisamente sotto soglia.

Informazione

PAGINA DI INTERMEZZO

La prova da 12 punti termina prima di questa pagina di intermezzo.

La prossima domanda rappresenta l'inizio della prova da 18 punti.

Domanda 6

Completo

Non valutata

Traccia da 18 punti:



È possibile rispondere alle 3 domande di questa pagina su Visual Studio Code, accessibile tramite la piattaforma CrownLabs (in fondo al testo di questa domanda).

Non riportare il codice nelle caselle di testo, se si sceglie di usare questa soluzione.

Descrizione del problema

È dato un mazzo da 52 carte, in cui per ognuno dei 4 semi (cuori, quadri, fiori e picche) sono presenti 13 carte di 13 valori:

carte numeriche da 1 a 10, più 3 figure (fante, donna e re). In questo contesto, per semplicità, si può supporre di attribuire alle tre figure i valori numerici 11 (fante), 12 (donna), 13 (re).

Dal mazzo viene selezionato un insieme S di carte, di cui si ha l'elenco (una lista concatenata).

Si vuole realizzare un'opportuna struttura dati, adatta a risolvere un problema di verifica e uno di ottimizzazione, come descritto nel seguito.

Attenzione: l'efficienza/complessità delle funzioni realizzate sarà oggetto di valutazione.

Aprire solo la piattaforma Crownlabs della traccia scelta, non entrambe!

La risposta a questa domanda è stata inserita tramite un'istanza Crownlabs ed è visibile sul Portale della Didattica sulla sezione **Consegna Elaborati** di questo insegnamento.

Domanda 7

Risposta non data

Punteggio max.: 4,00

Struttura dati

Si definisca un adeguato tipo **Card** in grado di rappresentare, come quasi ADT, una carta da gioco: una carta deve essere caratterizzata da una stringa (il nome per esteso: es. otto, fante, ...), il seme (codificato mediante un tipo enum), il valore (un numero da 1 a 13).

Si definiscano i tipi necessari per realizzare come ADT di prima classe una lista concatenata di carte: sia **CardLIST** il nome del tipo ADT. Il tipo ADT viene usato per rappresentare sia un insieme che una sequenza di carte. Per eventuali operazioni sulle liste, se lo si ritiene opportuno, è possibile utilizzare (scrivendone unicamente il prototipo) funzioni (standard) spiegate a lezione. Se sono necessarie altre funzioni, vanno realizzate.

Si definisca infine un tipo ADT di prima classe **SOL**, in grado di contenere due liste di carte che rappresentino il risultato del problema di ottimizzazione (due sequenza di carte di pari lunghezza). Si può supporre di realizzare un solo modulo, che esporti tutti gli ADT richiesti, avendo come client il main (non richiesto).

Domanda 8

Risposta non data

Punteggio max.: 4,00

Problema di verifica

Sono dati S_1 e S_2 , due sottoinsiemi non vuoti di S privi di intersezione (quindi $S_1 \cap S_2 = \{\}$): in pratica si tratta di insiemi di carte estratte in sequenza da S , prima S_1 , poi S_2 (preso dalla carte rimaste dopo aver tolto S_1).

Si scriva una funzione che, dati S_1 e S_2 , verifichi che per ogni carta di uno dei due insiemi esista almeno una carta dello stesso valore (ma di seme diverso) nell'altro insieme.

La funzione abbia prototipo

```
int checkCardLists(CardLIST S1, CardLIST S2);
```

Domanda 9

Risposta non data

Punteggio max.: 10,00

Problema di ricerca e ottimizzazione

Dall'insieme S si vogliono estrarre due sequenze (catene ordinate di carte, anche queste prive di carte in comune, non necessariamente utilizzando tutte le carte in S) della stessa lunghezza, tali da rispettare i vincoli seguenti:

- A. in ognuna delle due sequenze, due carte consecutive devono
- essere di seme diverso
 - avere valore che differisce al più di 2
 - le figure devono essere (in quantità) almeno la metà (divisione tra interi) dei numeri
- B. Gli insiemi della carte nelle due sequenze devono rispettare il criterio del problema precedente: per ogni carta di una delle sequenze deve esistere almeno una carta dello stesso valore (ma di seme diverso) nell'altra sequenza.

Si scriva una funzione che, dato l'insieme S , determini le due sequenze ottime, cioè quelle di lunghezza massima che rispettano i vincoli sopra elencati. In caso di uguaglianza è sufficiente ritornare una delle soluzioni ottime. La funzione abbia prototipo

```
SOL bestSequences (CardLIST S);
```

ATTENZIONE: È **necessario** indicare e motivare in modo esplicito il **modello del calcolo combinatorio** adottato. Nella funzione ricorsiva, un eventuale pruning va effettuato usando una funzione **prune**, che (ricevuti i parametri necessari) ritorni un valore vero/falso per indicare il taglio oppure l'effettuazione della chiamata ricorsiva. In modo simile si usi una funzione **checkSol**, che nel caso terminale verifichi l'accettabilità di una soluzione.